

Integração Supabase Compartilhado

Hackanation 2026 · TokenNation · 28/05/2026

Sumário

- 1. Contexto e modelo de compartilhamento
- 2. Infraestrutura do projeto Supabase
- 3. Tabelas e enums reutilizáveis
- 4. Edge Functions disponíveis (contratos)
- 5. Fluxo de autenticação (wallet Phantom → JWT)
- 6. Padrões obrigatórios (CORS, RLS, naming)
- 7. Escopo do schema próprio (Chain Oil)
- 8. Edge Function compartilhada planejada (usdc-to-pix)
- 9. Setup local e variáveis de ambiente
- 10. Endpoints completos com exemplos

1. Contexto e modelo de compartilhamento

Os projetos Uber Money e Chain Oil compartilham um único projeto Supabase (`qvoytjrfuyeammxsuwtx`). A estratégia adotada é **"infra base compartilhada, schemas isolados"**: tabelas de identidade, auth, documentos e payouts ficam no schema `public` (comuns aos dois); recursos específicos de cada projeto ficam em schemas próprios (`public` default para Uber Money e `chainoil` para Chain Oil).

Isso elimina duplicação de:

- Edge Function de autenticação Solana (`wallet-auth`)
- OCR via Claude Vision (`process-document`)
- Webhook do Woovi (`woovi-webhook`)
- Tabelas de usuários e nonces
- Bucket de Storage para documentos enviados
- Secrets (Anthropic API key, Woovi API key, JWT secret)

Plano v9 (Tainan, TG msg 6554, 27/05 12:25Z) — Q29: "1 Edge fn off-ramp compartilhada — README explícito sobre reuso deliberado." Esta arquitetura cumpre Q29 ao pé da letra.

2. Infraestrutura do projeto Supabase

Item	Valor
Project Reference	<code>qvoytjrfuyeammxsuwtx</code>
Region	East US (Ohio)
URL base	<code>https://qvoytjrfuyeammxsuwtx.supabase.co</code>
Edge Functions URL	<code>{URL}/functions/v1/{name}</code>
JWT issuer interno	<code>{URL}/auth/v1</code>
Storage bucket primário	<code>documents</code> (privado)

2.1. Secrets já configurados

Secret	Uso
WALLET_JWT_SECRET	HS256 do JWT custom mintado por <code>wallet-auth</code>
WALLET_UUID_NAMESPACE	Namespace UUID v5 (derivação determinística <code>wallet-user_id</code>)
ANTHROPIC_API_KEY	Claude Vision (modelo claude-haiku-4-5)
WOOVI_API_KEY	AppID direto no header <code>Authorization</code> — SEM prefixo <code>Bearer</code> . Sandbox e prod usam o mesmo formato, AppIDs distintos.
WOOVI_API_URL	Por ambiente — sandbox: <code>https://api.woovi-sandbox.com/api/v1</code> · prod: <code>https://api.woovi.com/api/v1</code>
WOOVI_WEBHOOK_SECRET	HMAC-SHA256 dos webhooks Woovi
WOOVI_MODE	<code>mock</code> <code>prod</code> — controla fallback do Pix
PAYOUT_MAX_BRL	Cap por payout (default 10, ajustável por projeto)
SOLANA_ADMIN_KEYPAIR_JSON	Array JSON da admin keypair Solana devnet
SOLANA_RPC_URL	Default <code>https://api.devnet.solana.com</code> (Helius como failover)
PROGRAM_ID	ID do programa Anchor (Uber Money) — não usado por Chain Oil

Nota importante: Supabase bloqueia secrets com prefixo `SUPABASE_*`. Por isso o JWT secret tem nome `WALLET_JWT_SECRET` (não `SUPABASE_JWT_SECRET`). O valor é a string raw do dashboard (não decodificar base64).

Errata Woovi (atualizada 28/05 12h): NÃO usar `https://api.openpix.com.br/api/v1` — domínio legacy, retorna "appID inválido" com AppIDs novos. URLs corretas:

- Sandbox: `https://api.woovi-sandbox.com/api/v1`
- Prod: `https://api.woovi.com/api/v1`

Auth: AppID direto em `Authorization` **SEM** prefixo `Bearer`. Confirmado contra spec OpenAPI oficial e smoke real em 28/05.

3. Tabelas e enums reutilizáveis

3.1. Schema `public` (compartilhado)

Tabela	Pode usar?	Observações
<code>users</code>	SIM	id (UUID), wallet (TEXT unique), cpf_pepper (bytea — só Uber Money usa), created_at
<code>nonces</code>	SIM	Auth flow ed25519 (consumido por <code>wallet-auth</code>)
<code>documents</code>	SIM	id, user_id, kind (enum), storage_url, ocr_data (JSONB), created_at. Constraint: UNIQUE (user_id, kind)
<code>cashout_intents</code>	SIM	Multi-source: <code>source ENUM ('uber_money', 'chain_oil')</code> . UNIQUE em (source, client_intent_id). Saga state para off-ramp USDC + Pix.
<code>payouts</code>	PARCIAL	Atualmente acoplada a <code>loans</code> (FK). Use <code>cashout_intents</code> para Chain Oil.
<code>loans</code> , <code>loan_requests</code> , <code>score_snapshots</code>	NÃO	Específicas do fluxo de microcrédito Uber Money

3.2. Enums relevantes

Enum	Valores atuais	Extensão para Chain Oil
document_kind	'cnh', 'print_earnings'	Adicionar 'pet_bottle', 'operator_id' via ALTER TYPE ADD VALUE
cashout_source	'uber_money', 'chain_oil'	Já contempla ambos
cashout_status	pending_signature, usdc_received, pix_dispatched, pix_confirmed, pix_failed_refund_due, failed	Genérico para ambos os projetos
pix_key_type	'cpf', 'email', 'phone', 'evp'	Sem alteração necessária

3.3. Storage bucket documents

Bucket privado. Acesso via Edge Function (service-role). Convenção de path:

```
documents/{user_id}/cnh-{timestamp}.jpg      # Uber Money
documents/{user_id}/print-{timestamp}.jpg     # Uber Money
documents/{user_id}/pet-{collection_id}.jpg   # Chain Oil (sugerido)
documents/{user_id}/operator-id-{timestamp}.jpg # Chain Oil (sugerido)
```

4. Edge Functions disponíveis (contratos)

4.1. POST /functions/v1/wallet-auth REUSE

Autenticação Solana ed25519. Duas ações:

ACTION: GET_NONCE

```
// Request
{
  "action": "get_nonce",
  "wallet": "<solana_pubkey_base58>"
}

// Response 200
{
  "nonce": "<hex_32>",
  "message": "uber-money:<nonce>"
}
```

ACTION: VERIFY

```
// Request
{
  "action": "verify",
  "wallet": "<solana_pubkey_base58>",
  "nonce": "<mesmo_nonce>",
  "signature": "<ed25519_signature_base58>"
}

// Response 200
{
  "user_id": "<uuid_v5(wallet, WALLET_UUID_NAMESPACE)>",
  "wallet": "<pubkey>",
  "access_token": "<jwt_hs256>"
}
```

JWT mintado: claims mínimos {aud:'authenticated', role:'authenticated', iss:'.../auth/v1', sub:user_id, email:user_id+'@wallet.local', app_metadata:{wallet,provider}, user_metadata:{wallet}, iat, exp} . **NÃO inclui session_id** (GoTrue rejeita session fake em admin.auth.getUser). Expiração padrão: 1h.

4.2. POST /functions/v1/process-document REUSE

Upload + OCR via Claude Vision. Requer Authorization Bearer com JWT do wallet-auth .

```
// Request
{
  "kind": "cnh" | "print_earnings" | "pet_bottle" | "operator_id",
  "imageBase64": "<base64 sem prefixo data:>",
  "mediaType": "image/jpeg" | "image/png" | "image/webp"
}

// Response 200
{
  "document_id": "<uuid>",
  "kind": "<kind>",
  "ocr_data": { /* depende do prompt configurado por kind */ }
}
```

Limites: 5 MB de base64 (~3.75 MB binário). Para novos kinds (`pet_bottle` , `operator_id`), a função atual precisa receber um prompt específico — recomendado abrir helper em `_shared/vision-prompts.ts` indexado por kind.

4.3. POST /functions/v1/woovi-webhook REUSE

Endpoint público (não autenticado por JWT — validado por HMAC-SHA256 do header `x-openpix-signature` com `WOОВI_WEBHOOK_SECRET`). Recebe eventos `OPENPIX:CHARGE_COMPLETED` e atualiza `payouts.status='confirmed'` pelo `correlationId` .

Para Chain Oil: o webhook deve atualizar `cashout_intents.status='pix_confirmed'` filtrando por `(source='chain_oil', client_intent_id=correlationId)` . A função precisa de patch para suportar a tabela `cashout_intents` além de `payouts` .

4.4. Edges específicas do Uber Money NÃO REUSAR

Função	Por que não reusar
<code>request-loan</code>	Lógica de score de motorista + validações Uber-specific
<code>request-payout</code>	Acoplada a <code>loans</code> + Anchor program ID do Uber Money
<code>parse-uber-print</code>	OCR específico do print do app Uber

5. Fluxo de autenticação (wallet Phantom → JWT)

Padrão "Sign-In with Solana" (SIWS) sem session GoTrue tradicional:

- Front pega nonce: `POST wallet-auth {action:'get_nonce', wallet}`
- Wallet assina `message` retornado via `signMessage(TextEncoder.encode(message))`
- Front verifica: `POST wallet-auth {action:'verify', wallet, nonce, signature}`
- Front armazena `access_token` em memória (ou Store) — NÃO usar `supabase.auth.setSession()` como fonte da verdade (rejeita silenciosamente sem refresh token válido)
- Toda chamada subsequente a Edge Functions: `Authorization: Bearer <access_token>`
- Edges validam via `admin.auth.getUser(token)` ou validação manual com `jose.jwtVerify`

5.1. Helper recomendado no front

```
// src/lib/api.ts (extraído do Uber Money)
let _walletAccessToken: string | null = null
export const setWalletAccessToken = (t: string | null) => { _walletAccessToken = t }

export async function authedFetch(path: string, body: unknown) {
  const token = _walletAccessToken
  || (await supabase().auth.getSession()).data.session?.access_token
  || SUPABASE_ANON_KEY
  return fetch(`${SUPABASE_URL}/functions/v1/${path}`, {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
      Authorization: `Bearer ${token}`,
    },
    body: JSON.stringify(body),
  })
}
```

6. Padrões obrigatórios

6.1. CORS

Reusar `supabase/functions/_shared/cors.ts` . Origens permitidas configuradas via secret `ALLOWED_ORIGINS` (CSV).

Webhooks (server-to-server) usam `handleOptionsOpen` + `jsonOpen`.

6.2. RLS (Row Level Security)

Padrão deny-all-client em tabelas sensíveis. Acesso somente via Edge Function com service-role:

```
ALTER TABLE chainoil.<table> ENABLE ROW LEVEL SECURITY;
CREATE POLICY chainoil_deny_client ON chainoil.<table>
  FOR ALL USING (false) WITH CHECK (false);
```

6.3. Idempotência

Para operações que envolvem dinheiro (Pix, mint/burn on-chain), gerar `client_intent_id` UUID no frontend por sessão. UNIQUE constraint na tabela impede execução dupla por double-click ou retry de network.

6.4. Naming de Edge Functions

Sugestão para evitar colisão e facilitar leitura: prefixar com nome do projeto quando exclusivas (`chainoil-register-collection` , `chainoil-mint-burn-signer`). Funções genuinamente compartilhadas ficam sem prefixo (`usdc-to-pix` , `wallet-auth`).

6.5. Migrations

Numeração sequencial global. Atual:

- `0001_init.sql` — users, nonces, documents, payouts, enums base
- `0002_nonces_policy.sql` — RLS nonces
- `0003_storage.sql` — bucket documents + policies
- `0004_security_hardening.sql` — RLS constraints anti-race
- `0005_uber_v2_cpf_hash.sql` — específico Uber Money (cpf_pepper)
- `0006_cashout_intents.sql` — multi-source off-ramp

Próximas migrations Chain Oil: `0007_chainoil_schema.sql` , `0008_chainoil_<...>.sql` , etc.

7. Escopo do schema próprio (Chain Oil)

7.1. Sugestão de migration inicial

```
-- 0007_chainoil_schema.sql
CREATE SCHEMA IF NOT EXISTS chainoil;

ALTER TYPE document_kind ADD VALUE IF NOT EXISTS 'pet_bottle';
ALTER TYPE document_kind ADD VALUE IF NOT EXISTS 'operator_id';

CREATE TABLE chainoil.collections (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  operator_id       UUID NOT NULL REFERENCES users(id) ON DELETE RESTRICT,
  citizen_phone     TEXT NOT NULL,
  liters            NUMERIC(6,2) NOT NULL CHECK (liters > 0),
  photo_url         TEXT NOT NULL,
  impact_score      INTEGER,
  mint_tx           TEXT,
  burn_tx           TEXT,
  pix_intent_id     UUID,
  created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_chainoil_collections_operator ON chainoil.collections (operator_id);
CREATE INDEX idx_chainoil_collections_phone   ON chainoil.collections (citizen_phone);

ALTER TABLE chainoil.collections ENABLE ROW LEVEL SECURITY;
CREATE POLICY chainoil_collections_deny_client ON chainoil.collections
  FOR ALL USING (false) WITH CHECK (false);

-- Próximas tabelas conforme necessidade:
-- chainoil.operators, chainoil.oil_token_log, chainoil.esg_metrics, ...
```

7.2. Edges específicas a criar

Edge	Responsabilidade
chainoil-register-collection	Recebe {operator, citizen_phone, liters, photo}. Grava em chainoil.collections . Aciona chainoil-mint-burn-signer . Aciona usdc-to-pix (Q29 compartilhada).
chainoil-mint-burn-signer	Server-side: assina spl-token mint-to + spl-token burn via admin keypair (mesma keypair Solana ou nova; sugerido nova para isolar autoridade de mint do COT).

8. Edge Function compartilhada planejada: usdc-to-pix

Implementação prevista para 28/05 (pelo time Uber Money). Contrato:

```
POST /functions/v1/usdc-to-pix
Authorization: Bearer <jwt>

// Request
{
  "source": "uber_money" | "chain_oil",
  "intentId": "<uuid_v4_gerado_no_front>",
  "amountBRL": 2.40,
  "pixKeyType": "cpf" | "phone" | "email" | "evp",
  "pixKey": "+5511999999999",
  "solanaSignature": "<opcional, tx user=tesouraria se houver USDC return>"
}

// Response 200
{
  "intentId": "<mesmo uuid>",
  "status": "pending" | "mock_8s" | "confirmed",
  "pixId": "<woovi correlation id>",
  "amountBRL": 2.40,
  "mode": "prod" | "sandbox" | "mock"
}

// Response 409 (idempotência)
{
  "intentId": "<uuid>",
  "status": "already_dispatched",
  "pixId": "<existente>"
}
```

Comportamento interno:

1. Valida JWT do chamador (qualquer user autenticado).
2. UPSERT em cashout_intents com (source, intentId) UNIQUE.
3. Se houver solanaSignature : valida on-chain via getParsedTransaction (destino = tesouraria correta por source).
4. Em modo prod : POST {WOОВI_API_URL}/charge com Bearer auth.
5. Em modo mock : agenda update setTimeout(8s) marcando pix_confirmed .
6. Retorna intent + status atual.

9. Setup local e variáveis de ambiente

9.1. CLI Supabase

```
# Linux: extrair release oficial em ~/.local/share/supabase (contém binário companion)
mkdir -p "$HOME/.local/share/supabase"
curl -sL https://github.com/supabase/cli/releases/download/v2.101.0/supabase_2.101.0_linux_amd64.tar.gz \
| tar -xzf - -C "$HOME/.local/share/supabase"
export PATH="$HOME/.local/share/supabase:$PATH"
# Persistente no ~/.bashrc:
export SUPABASE_GO_BINARY="$HOME/.local/share/supabase/supabase-go"
```

9.2. Vincular ao projeto

```
supabase login          # browser opens
supabase link --project-ref qvoytjrfuyeammxsuwtx
supabase db pull        # sync schema atual local
```

9.3. Variáveis de ambiente (.env do projeto local)

```
# Front (Vite)
VITE_SUPABASE_URL=https://qvoytjrfuyeammxsuwx.supabase.co
VITE_SUPABASE_ANON_KEY=<anon_key_do_dashboard>
VITE_SOLANA_CLUSTER=devnet
VITE_HELIUS_RPC_URL=<opcional, https://devnet.helius-rpc.com/?api-key=...>

# Edge runtime (já setadas como secret no Supabase para wallet-auth, woovi-webhook,
# process-document — não precisa setar novamente)
```

9.4. Deploy de edge nova

```
supabase functions deploy chainoil-register-collection \
  --project-ref qvoytjrfuyeammxsuwx
```

9.5. Aplicar migrations

```
supabase db push --linked --password "<db_password>"
```

10. Endpoints com exemplos curl

10.1. Auth flow completo

```
# 1. Pega nonce
curl -X POST https://qvoytjrfuyeammxsuwx.supabase.co/functions/v1/wallet-auth \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $ANON_KEY" \
  -d '{"action": "get_nonce", "wallet": "<PUBKEY_BASE58>"}'

# Response: { "nonce": "...", "message": "uber-money:..." }

# 2. Assina (cliente faz com Phantom signMessage)
# signature_base58 = ed25519(secret_key, message)

# 3. Verifica
curl -X POST https://qvoytjrfuyeammxsuwx.supabase.co/functions/v1/wallet-auth \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $ANON_KEY" \
  -d '{"action": "verify", "wallet": "<PUBKEY>", "nonce": "<NONCE>", "signature": "<SIG_B58>"}'

# Response: { "user_id": "...", "wallet": "...", "access_token": "<JWT>" }
TOKEN=<access_token retornado>
```

10.2. Upload de documento (OCR)

```
curl -X POST https://qvoytjrfuyeammxsuwx.supabase.co/functions/v1/process-document \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $TOKEN" \
  -d '{
    "kind": "pet_bottle",
    "imageBase64": "<base64>",
    "mediaType": "image/jpeg"
  }'

# Response: { "document_id": "...", "kind": "pet_bottle", "ocr_data": {...} }
```

10.3. Inserir cashout_intent (Chain Oil — quando edge usdc-to-pix estiver pronta)

```
curl -X POST https://qvoytjrfuyeammxsuwx.supabase.co/functions/v1/usdc-to-pix \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $TOKEN" \
  -d '{
    "source": "chain_oil",
    "intentId": "550e8400-e29b-41d4-a716-446655440000",
    "amountBRL": 2.40,
    "pixKeyType": "phone",
    "pixKey": "+5511999999999"
  }'
```

11. Pontos de atenção

- **Conflito em** `document_kind` **enum:** adicionar novos valores requer migration. Coordenar para não conflitar.
- **Storage paths:** usar prefixo de projeto para evitar colisão (`chainoil/<user_id>/...`).

- **WOOVI_WEBHOOK_SECRET** é único para o projeto. Ambos os fluxos (Uber Money e Chain Oil) recebem no mesmo endpoint `woovi-webhook` ; o handler precisa distinguir por `correlationId` ou consultar `cashout_intents.source` .
- **JWT custom mintado** não tem entrada em `auth.sessions` ; refresh não funciona — usar token até expirar (1h) e refazer auth flow.
- **RLS:** tabelas do schema `chainoil` devem ter deny-all-client por default. Acesso somente via service-role na Edge.
- **Mainnet:** esta config é toda devnet. Mainnet exige program ID novo, mint authority nova e revisão de RLS.